

## 70 - Implementing login functionality

### Login Functionality in ASP.NET Core

LoginViewModel

Login View

Login Actions - Account Controller

Където Login Actions = за Get + Post

### LoginViewModel

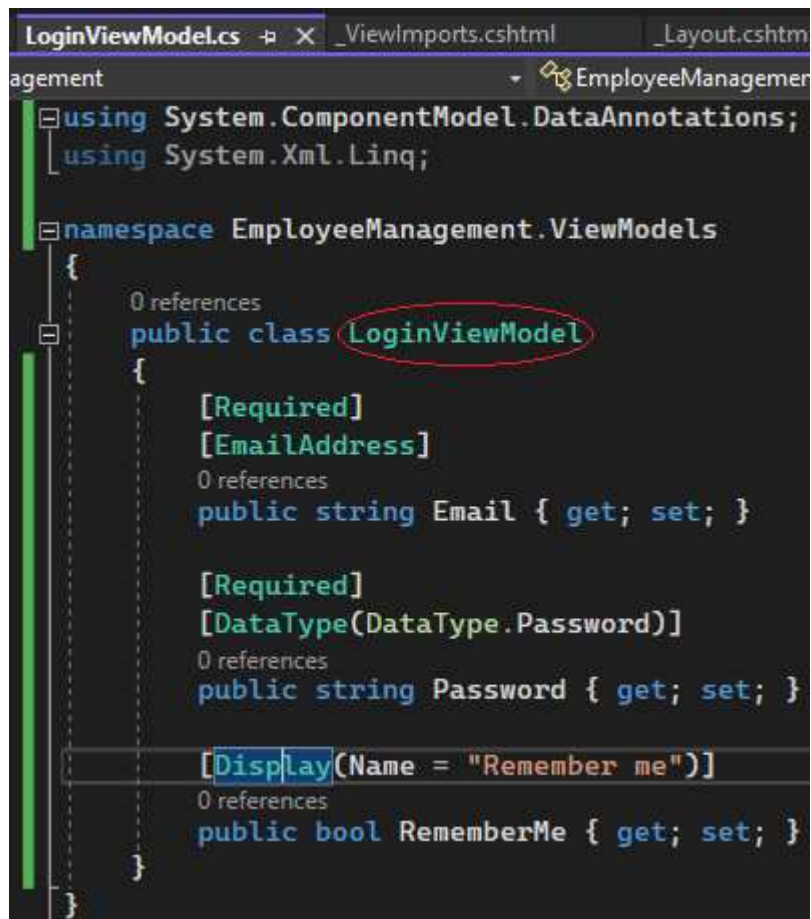
```
public class LoginViewModel
{
    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Required]
    [DataType(DataType.Password)]
    public string Password { get; set; }

    [Display(Name = "Remember me")]
    public bool RememberMe { get; set; }
}
```

ViewModels folder -> add -> New item -> class -> LoginViewModel.cs

Необходими са ни 3 свойства: Email, Password, Remember we:



```
using System.ComponentModel.DataAnnotations;
using System.Xml.Linq;

namespace EmployeeManagement.ViewModels
{
    public class LoginViewModel
    {
        [Required]
        [EmailAddress]
        public string Email { get; set; }

        [Required]
        [DataType(DataType.Password)]
        public string Password { get; set; }

        [Display(Name = "Remember me")]
        public bool RememberMe { get; set; }
    }
}
```

```
using System.ComponentModel.DataAnnotations;
using System.Xml.Linq;
```

```
namespace EmployeeManagement.ViewModels
{
    public class LoginViewModel
    {
        [Required]
        [EmailAddress]
        public string Email { get; set; }

        [Required]
        [DataType(DataType.Password)]
        public string Password { get; set; }

        [Display(Name = "Remember me")]
        public bool RememberMe { get; set; }
    }
}
```

След това създаваме Login view, в който клас `LoginViewModel` ще бъде модел.  
Views/Account-> Add -> New item -> Razor view -> Login.cshtml

```

Login.cshtml* X LoginViewModel.cs _ViewImports.cshtml _Layout.cshtml AccountController
@model LoginViewModel

@{
    ViewBag.Title = "User Login";
}

<h1>User Login</h1>

<div class="row">
    <div class="col-md-12">
        <form method="post">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="Email"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Password"></label>
                <input asp-for="Password" class="form-control" />
                <span asp-validation-for="Password" class="text-danger"></span>
            </div>
            <div class="form-group">
                <div class="checkbox">
                    <label asp-for="RememberMe">
                        <input asp-for="RememberMe" />
                        @Html.DisplayNameFor(m => m.RememberMe)
                    </label>
                    bool
                </div>
            </div>
        </form>
    </div>
</div>

```

+ Login bnt -> Post request

@model LoginViewModel

```

@{
    ViewBag.Title = "User Login";
}

```

<h1>User Login</h1>

```

<div class="row">
    <div class="col-md-12">
        <form method="post">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="Email"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Password"></label>
                <input asp-for="Password" class="form-control" />
                <span asp-validation-for="Password" class="text-danger"></span>
            </div>
            <div class="form-group">
                <div class="checkbox">
                    <label asp-for="RememberMe">
                        <input asp-for="RememberMe" />
                        @Html.DisplayNameFor(m => m.RememberMe)
                    </label>
                </div>
            </div>
        </form>
    </div>
</div>

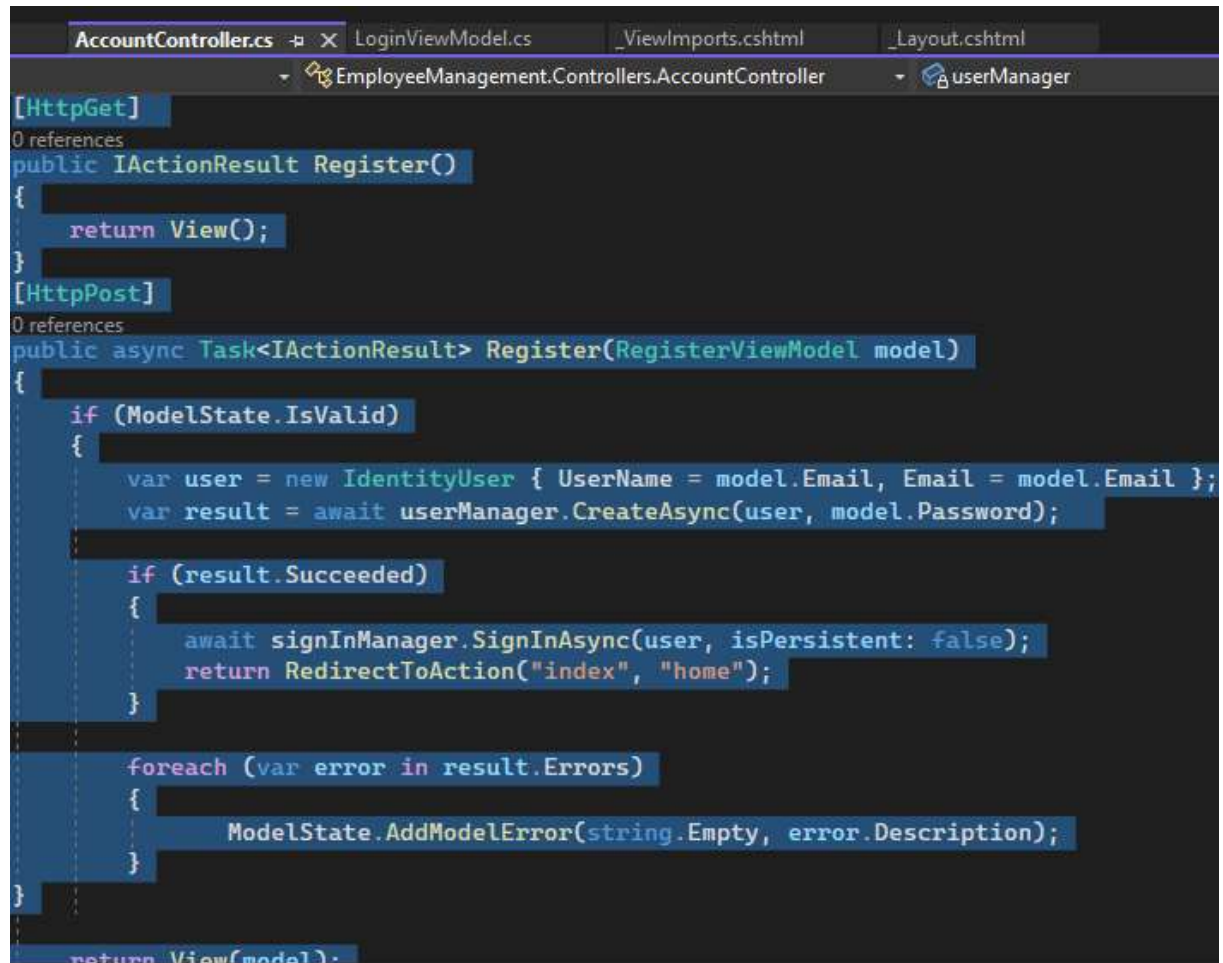
```

```

        </label>
    </div>
</div>
<button type="submit" class="btn btn-primary">Login</button>
</form>
</div>
</div>

```

Понеже Login много наподобява Register, може да го копираме и изменим:



```

AccountController.cs X LoginViewModel.cs _ViewImports.cshtml _Layout.cshtml
EmployeeManagement.Controllers.AccountController userManager

[HttpGet]
0 references
public IActionResult Register()
{
    return View();
}

[HttpPost]
0 references
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (ModelState.IsValid)
    {
        var user = new IdentityUser { UserName = model.Email, Email = model.Email };
        var result = await userManager.CreateAsync(user, model.Password);

        if (result.Succeeded)
        {
            await signInManager.SignInAsync(user, isPersistent: false);
            return RedirectToAction("index", "home");
        }

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError(string.Empty, error.Description);
        }
    }

    return View(model);
}

```

Вместо userManager -> signInManager.

Не създаваме нов потребител!

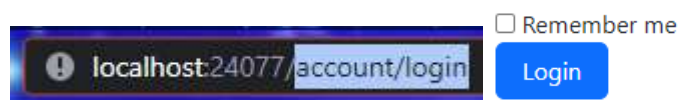
```
AccountController.cs* LoginViewModel.cs _ViewImports.cshtml _Layout.cshtml
EmployeeManagement.Controllers.AccountController Login(LoginViewMod

[HttpGet]
0 references
public IActionResult Login()
{
    return View();
}

[HttpPost]
0 references
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync();
    }
    if
```

Проверката за вписване изисква подаване на 4 properties: име, в случая имейл, парола, бисквитки (IsPersistent) в булев формат, които в случая идват от RememberMe, булева true/false за действие при грешка при вписване. Ако вписването е успешно, препращаме потребителя в начална страница, ако не – извежда съобщение за грешка и презарежда страницата с данните.

```
[HttpPost]
0 references
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,
                                                             model.RememberMe, false);
        if (result.Succeeded)
        {
            return RedirectToAction("index", "home");
        }
        else ModelState.AddModelError(string.Empty, "Invalid Login attempt");
    }
    return View(model);
}
```



```
[HttpGet]
public IActionResult Login()
{
    return View();
}

[HttpPost]
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync(model.Email,
model.Password,
```

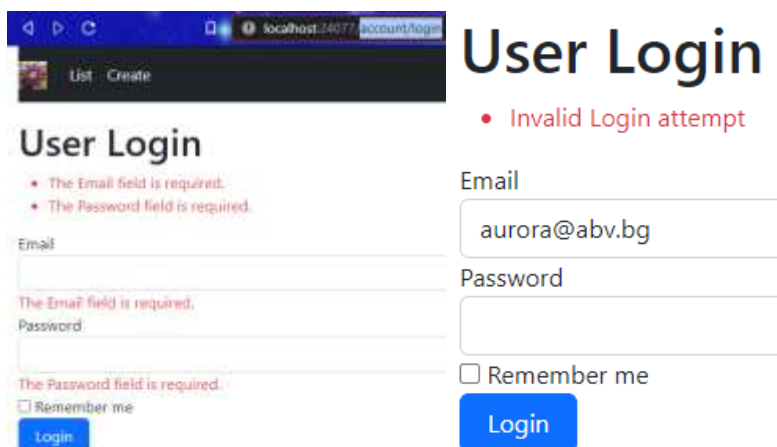


```

model.RememberMe, false);
    if (result.Succeeded)
    {
        return RedirectToAction("index", "home");
    }

    ModelState.AddModelError(string.Empty, "Invalid Login attempt");
}
return View(model);
}

```



## User Login

- Invalid Login attempt

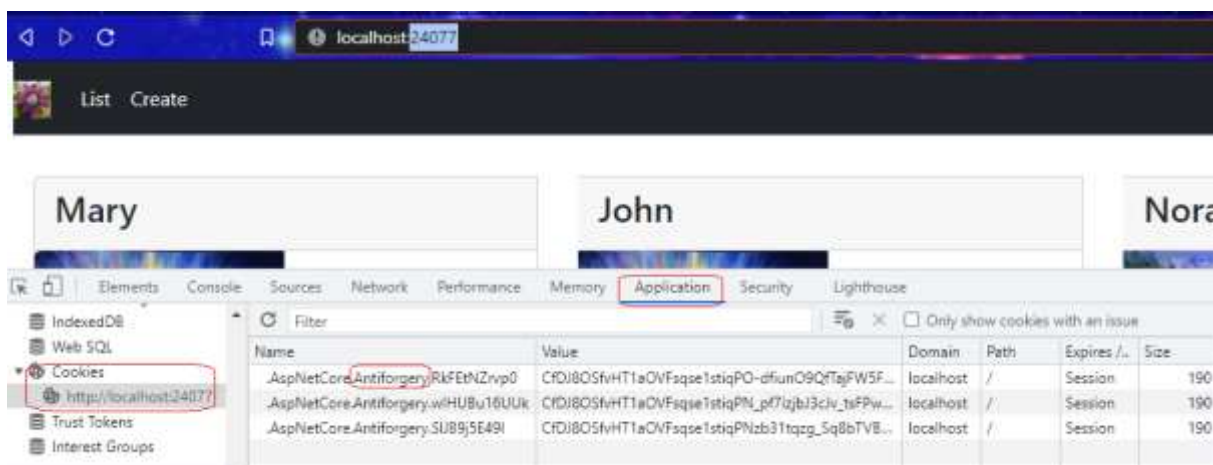
- The Email field is required.
- The Password field is required.

Email:

Password:

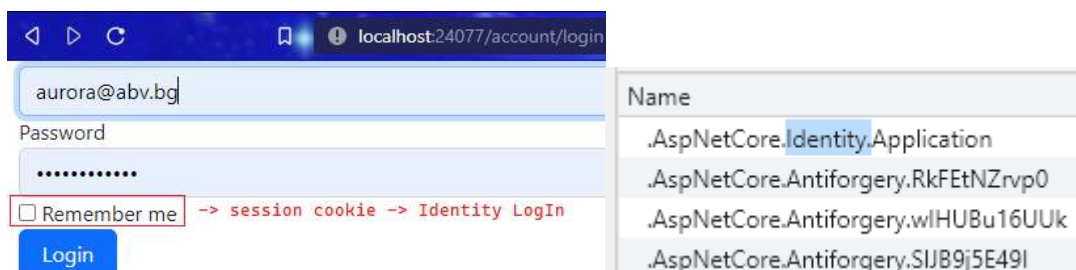
☐ Remember me

Нека разберем разликите между 2-та вида бисквитки:



| Name                                | Value                                          | Domain    | Path | Expires / | Size |
|-------------------------------------|------------------------------------------------|-----------|------|-----------|------|
| .AspNetCore.Antiforgery.RkFetNZrvp0 | CFDj80SfvHT1aOVFsqe1stiqPO-dfunO9QtajFW5F...   | localhost | /    | Session   | 190  |
| .AspNetCore.Antiforgery.wIHUBu16UUk | CFDj80SfvHT1aOVFsqe1stiqPM_p7Izbi3cIv_1sFPw... | localhost | /    | Session   | 190  |
| .AspNetCore.Antiforgery.SIJB9j5E49I | CFDj80SfvHT1aOVFsqe1stiqPNzb31tqzg_Sq8bTVB...  | localhost | /    | Session   | 190  |

Developer Tools -> Application -> Cookies -> имаме само antiforgery (срещу фалшификати).



aurora@abv.bg

Password:

☒ Remember me -> session cookie -> Identity Login

| Name                                |
|-------------------------------------|
| .AspNetCore.Identity.Application    |
| .AspNetCore.Antiforgery.RkFetNZrvp0 |
| .AspNetCore.Antiforgery.wIHUBu16UUk |
| .AspNetCore.Antiforgery.SIJB9j5E49I |

С тази бисквитка всеки път се подават данните от браузъра до сървъра за вписване. Понеже не чекнахме Remember me -> session cookie, която незабавно се губи когато се затвори подпрозореца на браузъра. Нека го проверим:

Save password?

Username

Password



Не запомням паролата в браузъра, при Logout, бисквитка Identity се унищожава. Вписвам отново, създава се, но при мен бисквитката за Identity стои независимо дали затварям подпрозорец, целия браузър, дали стартирам приложението наново. В туториъла обаче бисквитките изчезват и register-login се занулява. Запазва се само при маркиране на Remember me. Ако искаме да занулим постоянна persistent cookie, просто даваме Logout, така че при зареждане на браузъра наново, ще трябва да се впишем.

## Login View

```
@model LoginViewModel
<form method="post">
  <div asp-validation-summary="All" class="text-danger"></div>
  <div class="form-group">
    <label asp-for="Email"></label>
    <input asp-for="Email" class="form-control" />
    <span asp-validation-for="Email" class="text-danger"></span>
  </div>
  <div class="form-group">
    <label asp-for="Password"></label>
    <input asp-for="Password" class="form-control" />
    <span asp-validation-for="Password" class="text-danger"></span>
  </div>
  <div class="form-group">
    <div class="checkbox">
      <label asp-for="RememberMe">
        <input asp-for="RememberMe" />
        @Html.DisplayNameFor(m => m.RememberMe)
      </label>
    </div>
  </div>
  <button type="submit" class="btn btn-primary">Login</button>
</form>
```

# Login Actions

```
[HttpGet]
public IActionResult Login()
{
    return View();
}
```

```
[HttpPost]
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync(model.Email,
            model.Password, model.RememberMe, false);

        if (result.Succeeded)
        {
            return RedirectToAction("index", "home");
        }

        ModelState.AddModelError(string.Empty, "Invalid Login Attempt");
    }

    return View(model);
}
```